

Use case: GET /articles/{slug}

Objective: Retrieve an article representation identified by slug.

Actors: System

Preconditions: The slug path parameter is provided.

Main flow:

1. The controller invokes `articleQueryService.findBySlug(slug, user)` to locate the article.
2. If an `ArticleData` is present it is wrapped using `articleResponse` and returned with HTTP 200.
3. If no article is found the controller throws `ResourceNotFoundException`.

Postconditions: 200 OK with a response body mapping "article" to `ArticleData` when found.

Possible errors: `ResourceNotFoundException`

Related endpoint:

GET /articles/{slug}

Traceability: Controller=`ArticleApi.java` | Method=`article` | Mapping=`@GetMapping`

Request (Contract):

```
{
  "bodyType": null,
  "bodySchema": null,
  "parametros": {
    "path": {
      "slug": "string"
    }
  }
}
```

Response (Contract):

```
{
  "responseType": "HashMap(String,ArticleData)",
  "responseSchema": null
}
```

Use case: PUT /articles/{slug}

Objective: Update the article identified by slug with provided data.

Actors: Authenticated user

Preconditions: The slug path parameter and a valid UpdateArticleParam request body are provided; an authenticated user principal is present.

Main flow:

1. The controller looks up the article via `articleRepository.findBySlug(slug)` or throws `ResourceNotFoundException`.
2. The controller checks `AuthorizationService.canWriteArticle(user, article)` and throws `NoAuthorizationException` if not authorized.
3. The controller calls `articleCommandService.updateArticle(article, updateArticleParam)` to perform the update.
4. The controller retrieves the updated `ArticleData` via `articleQueryService.findBySlug(updatedArticle.getSlug(), user.get())` and returns it wrapped under "article" with HTTP 200.

Postconditions: 200 OK with a response body mapping "article" to the updated `ArticleData`.

Possible errors: `NoAuthorizationException`; `ResourceNotFoundException`

Related endpoint:

PUT /articles/{slug}

Traceability: Controller=ArticleApi.java | Method=updateArticle | Mapping=@PutMapping

Request (Contract):

```
{
  "bodyType": "UpdateArticleParam",
  "bodySchema": null,
  "parametros": {
    "path": {
      "slug": "string"
    }
  }
}
```

Response (Contract):

```
{
  "responseType": "HashMap(String,ArticleData)",
  "responseSchema": null
}
```

Use case: DELETE /articles/{slug}

Objective: Delete the article identified by slug.

Actors: Authenticated user

Preconditions: The slug path parameter is provided and an authenticated user principal is present.

Main flow:

1. The controller finds the article via `articleRepository.findBySlug(slug)` or throws `ResourceNotFoundException`.
2. The controller verifies `AuthorizationService.canWriteArticle(user, article)` and throws `NoAuthorizationException` if not authorized.
3. The controller removes the article via `articleRepository.remove(article)` and returns HTTP 204 No Content.

Postconditions: 204 No Content when deletion succeeds.

Possible errors: `NoAuthorizationException`; `ResourceNotFoundException`

Related endpoint:

DELETE /articles/{slug}

Traceability: Controller=ArticleApi.java | Method=deleteArticle | Mapping=@DeleteMapping

Request (Contract):

```
{
  "bodyType": null,
  "bodySchema": null,
  "parametros": {
    "path": {
      "slug": "string"
    }
  }
}
```

Response (Contract):

```
{
  "responseType": null,
  "responseSchema": null
}
```

Use case: POST /articles/{slug}/favorite

Objective: Mark the specified article as favorited by the authenticated user and return updated article data.

Actors: Authenticated user

Preconditions: The slug path parameter is provided and an authenticated user principal is present.

Main flow:

1. The controller looks up the Article via `articleRepository.findBySlug(slug)` or throws `ResourceNotFoundException`.
2. The controller creates an `ArticleFavorite(article.getId(), user.getId())` and saves it using `articleFavoriteRepository.save(articleFavorite)`.

3. The controller retrieves updated ArticleData via `articleQueryService.findBySlug(slug, user).get()` and returns it wrapped under "article" with HTTP 200.

Postconditions: 200 OK with a response body mapping "article" to the updated ArticleData.

Possible errors: ResourceNotFoundException

Related endpoint:

POST /articles/{slug}/favorite

Traceability: Controller=ArticleFavoriteApi.java | Method=favoriteArticle | Mapping=@PostMapping

Request (Contract):

```
{
  "bodyType": null,
  "bodySchema": null,
  "parametros": {
    "path": {
      "slug": "string"
    }
  }
}
```

Response (Contract):

```
{
  "responseType": "HashMap(String,ArticleData)",
  "responseSchema": null
}
```

Use case: DELETE /articles/{slug}/favorite

Objective: Remove the authenticated user's favorite relation for the specified article and return updated article data.

Actors: Authenticated user

Preconditions: The slug path parameter is provided and an authenticated user principal is present.

Main flow:

1. The controller finds the Article via `articleRepository.findBySlug(slug)` or throws ResourceNotFoundException.
2. The controller looks up an ArticleFavorite via `articleFavoriteRepository.find(article.getId(), user.getId())` and if present removes it.
3. The controller retrieves updated ArticleData via `articleQueryService.findBySlug(slug, user).get()` and returns it wrapped under "article" with HTTP 200.

Postconditions: 200 OK with a response body mapping "article" to the updated ArticleData.

Possible errors: ResourceNotFoundException

Related endpoint:

DELETE /articles/{slug}/favorite

Traceability: Controller=ArticleFavoriteApi.java | Method=unfavoriteArticle | Mapping=@DeleteMapping

Request (Contract):

```
{
  "bodyType": null,
  "bodySchema": null,
  "parametros": {
    "path": {
      "slug": "string"
    }
  }
}
```

Response (Contract):

```
{
  "responseType": "HashMap(String,ArticleData)",
  "responseSchema": null
}
```

Use case: POST /articles

Objective: Create a new article using provided payload.

Actors: Authenticated user

Preconditions: A valid NewArticleParam request body is provided and an authenticated user principal is present.

Main flow:

1. The controller calls articleCommandService.createArticle(newArticleParam, user) to create the Article.
2. The controller retrieves the created ArticleData via articleQueryService.findById(article.getId(), user).get() and returns it wrapped under "article" with HTTP 200.

Postconditions: 200 OK with a response body mapping "article" to the created ArticleData.

Possible errors: No evidence was found in the provided source code.

Related endpoint:

POST /articles

Traceability: Controller=ArticlesApi.java | Method=createArticle | Mapping=@PostMapping

Request (Contract):

```
{
  "bodyType": "NewArticleParam",
  "bodySchema": null,
  "parametros": {}
}
```

Response (Contract):

```
{
  "responseType": "HashMap(String,ArticleData)",
  "responseSchema": null
}
```

Use case: GET /articles/feed

Objective: Retrieve a personalized feed of articles for the authenticated user.

Actors: Authenticated user

Preconditions: An authenticated user principal is present; optional query parameters offset and limit may be provided.

Main flow:

1. The controller calls `articleQueryService.findUserFeed(user, new Page(offset, limit))` and returns the result with HTTP 200.

Postconditions: 200 OK with a paginated Page of ArticleData for the user.

Possible errors: No evidence was found in the provided source code.

Related endpoint:

GET /articles/feed

Traceability: Controller=ArticlesApi.java | Method=getFeed | Mapping=@GetMapping(path = "feed")

Request (Contract):

```
{
  "bodyType": null,
  "bodySchema": null,
  "parametros": {
    "query": {
      "offset": "int (default 0)",
      "limit": "int (default 20)"
    }
  }
}
```

Response (Contract):

```
{
  "responseType": "Page(ArticleData)",
}
```

```
"responseSchema": null
}
```

Use case: GET /articles

Objective: Retrieve recent articles optionally filtered by tag, author, or favorited user.

Actors: System

Preconditions: Optional query parameters offset, limit, tag, favorited, and author may be provided.

Main flow:

1. The controller calls `articleQueryService.findRecentArticles(tag, author, favoritedBy, new Page(offset, limit), user)` and returns the result with HTTP 200.

Postconditions: 200 OK with a paginated Page of ArticleData matching provided filters.

Possible errors: No evidence was found in the provided source code.

Related endpoint:

GET /articles

Traceability: Controller=ArticlesApi.java | Method=getArticles | Mapping=@GetMapping

Request (Contract):

```
{
  "bodyType": null,
  "bodySchema": null,
  "parametros": {
    "query": {
      "offset": "int (default 0)",
      "limit": "int (default 20)",
      "tag": "string (optional)",
      "favorited": "string (optional)",
      "author": "string (optional)"
    }
  }
}
```

Response (Contract):

```
{
  "responseType": "Page(ArticleData)",
  "responseSchema": null
}
```

Use case: POST /articles/{slug}/comments

Objective: Create a new comment for the specified article.

Actors: Authenticated user

Preconditions: The slug path parameter, a valid NewCommentParam request body are provided and an authenticated user principal is present.

Main flow:

1. The controller finds the Article via `articleRepository.findBySlug(slug)` or throws `ResourceNotFoundException`.
2. The controller constructs a new Comment with `newCommentParam.getBody()`, `user.getId()`, `article.getId()` and saves it via `commentRepository.save(comment)`.
3. The controller retrieves `CommentData` via `commentQueryService.findById(comment.getId(), user.getId())` and returns it wrapped under "comment" with HTTP 201.

Postconditions: 201 Created with a response body mapping "comment" to the created `CommentData`.

Possible errors: `ResourceNotFoundException`

Related endpoint:

POST /articles/{slug}/comments

Traceability: Controller=CommentsApi.java | Method=createComment | Mapping=@PostMapping

Request (Contract):

```
{
  "bodyType": "NewCommentParam",
  "bodySchema": null,
  "parametros": {
    "path": {
      "slug": "string"
    }
  }
}
```

Response (Contract):

```
{
  "responseType": "HashMap(String,CommentData)",
  "responseSchema": null
}
```

Use case: GET /articles/{slug}/comments

Objective: Retrieve all comments for the specified article.

Actors: System

Preconditions: The slug path parameter is provided.

Main flow:

1. The controller looks up the Article via `articleRepository.findBySlug(slug)` or throws `ResourceNotFoundException`.

2. The controller fetches comments via `commentQueryService.findByArticleId(article.getId(), user)` and returns them wrapped under "comments" with HTTP 200.

Postconditions: 200 OK with a response body mapping "comments" to a list of `CommentData`.

Possible errors: `ResourceNotFoundException`

Related endpoint:

GET /articles/{slug}/comments

Traceability: Controller=CommentsApi.java | Method=getComments | Mapping=@GetMapping

Request (Contract):

```
{
  "bodyType": null,
  "bodySchema": null,
  "parametros": {
    "path": {
      "slug": "string"
    }
  }
}
```

Response (Contract):

```
{
  "responseType": "HashMap(String, List(CommentData))",
  "responseSchema": null
}
```

Use case: DELETE /articles/{slug}/comments/{id}

Objective: Delete a specific comment from the specified article.

Actors: Authenticated user

Preconditions: The slug and id path parameters are provided and an authenticated user principal is present.

Main flow:

1. The controller finds the Article via `articleRepository.findBySlug(slug)` or throws `ResourceNotFoundException`.
2. The controller looks up the Comment via `commentRepository.findById(article.getId(), commentId)` or throws `ResourceNotFoundException`.
3. The controller checks `AuthorizationService.canWriteComment(user, article, comment)` and throws `NoAuthorizationException` if not authorized.
4. The controller removes the comment via `commentRepository.remove(comment)` and returns HTTP 204 No Content.

Postconditions: 204 No Content when deletion succeeds.

Possible errors: ResourceNotFoundException; NoAuthorizationException

Related endpoint:

DELETE /articles/{slug}/comments/{id}

Traceability: Controller=CommentsApi.java | Method=deleteComment |
Mapping=@RequestMapping(path = "{id}", method = RequestMethod.DELETE)

Request (Contract):

```
{
  "bodyType": null,
  "bodySchema": null,
  "parametros": {
    "path": {
      "slug": "string",
      "id": "string"
    }
  }
}
```

Response (Contract):

```
{
  "responseType": null,
  "responseSchema": null
}
```

Use case: GET /user

Objective: Return the currently authenticated user along with the token.

Actors: Authenticated user

Preconditions: An authenticated user principal is present and the Authorization header containing the bearer token is provided.

Main flow:

1. The controller extracts the bearer token from the Authorization header and calls `userService.findById(currentUser.getId()).get()` to obtain `UserData`.
2. The controller constructs `UserWithToken(userData, token)` and returns it wrapped under "user" with HTTP 200.

Postconditions: 200 OK with a response body mapping "user" to `UserWithToken`.

Possible errors: No evidence was found in the provided source code.

Related endpoint:

GET /user

Traceability: Controller=CurrentUserApi.java | Method=currentUser |
Mapping=@GetMapping

Request (Contract):

```
{
  "bodyType": null,
  "bodySchema": null,
  "parametros": {
    "headers": {
      "Authorization": "string (format: \"Bearer <token>\")"
    }
  }
}
```

Response (Contract):

```
{
  "responseType": "HashMap(String,UserWithToken)",
  "responseSchema": null
}
```

Use case: PUT /user

Objective: Update the profile of the currently authenticated user and return the updated user with token.

Actors: Authenticated user

Preconditions: An authenticated user principal is present, the Authorization header containing the bearer token is provided, and a valid UpdateUserParam request body is provided.

Main flow:

1. The controller invokes userService.updateUser(new UpdateUserCommand(currentUser, updateUserParam)) to apply updates.
2. The controller fetches fresh UserData via userQueryService.findById(currentUser.getId()).get(), constructs a UserWithToken with the bearer token, and returns it wrapped under "user" with HTTP 200.

Postconditions: 200 OK with a response body mapping "user" to the updated UserWithToken.

Possible errors: No evidence was found in the provided source code.

Related endpoint:

PUT /user

Traceability: Controller=CurrentUserApi.java | Method=updateProfile | Mapping=@PutMapping

Request (Contract):

```
{
  "bodyType": "UpdateUserParam",
  "bodySchema": null,
}
```

```
"parametros": {
  "headers": {
    "Authorization": "string (format: \"Bearer &token>\")"
  }
}
```

Response (Contract):

```
{
  "responseType": "HashMap(String,UserWithToken)",
  "responseSchema": null
}
```

Use case: GET /profiles/{username}

Objective: Retrieve profile information for the specified username.

Actors: System

Preconditions: The username path parameter is provided.

Main flow:

1. The controller calls `profileQueryService.findByUsername(username, user)` and if present returns the `ProfileData` wrapped under "profile" with HTTP 200.
2. If no profile is found the controller throws `ResourceNotFoundException`.

Postconditions: 200 OK with a response body mapping "profile" to `ProfileData` when found.

Possible errors: `ResourceNotFoundException`

Related endpoint:

GET /profiles/{username}

Traceability: Controller=ProfileApi.java | Method=getProfile | Mapping=@GetMapping

Request (Contract):

```
{
  "bodyType": null,
  "bodySchema": null,
  "parametros": {
    "path": {
      "username": "string"
    }
  }
}
```

Response (Contract):

```
{
  "responseType": "HashMap(String,ProfileData)",

```

```
"responseSchema": null
}
```

Use case: POST /profiles/{username}/follow

Objective: Create a follow relation from the authenticated user to the target user and return the updated profile.

Actors: Authenticated user

Preconditions: The username path parameter is provided and an authenticated user principal is present.

Main flow:

1. The controller looks up the target user via `userRepository.findByUsername(username)` or throws `ResourceNotFoundException`.
2. The controller creates and saves a `FollowRelation(user.getId(), target.getId())` via `userRepository.saveRelation`.
3. The controller retrieves updated `ProfileData` via `profileQueryService.findByUsername(username, user).get()` and returns it wrapped under "profile" with HTTP 200.

Postconditions: 200 OK with a response body mapping "profile" to the updated `ProfileData`.

Possible errors: `ResourceNotFoundException`

Related endpoint:

POST /profiles/{username}/follow

Traceability: Controller=ProfileApi.java | Method=follow | Mapping=@PostMapping(path = "follow")

Request (Contract):

```
{
  "bodyType": null,
  "bodySchema": null,
  "parametros": {
    "path": {
      "username": "string"
    }
  }
}
```

Response (Contract):

```
{
  "responseType": "HashMap(String,ProfileData)",
  "responseSchema": null
}
```

Use case: DELETE /profiles/{username}/follow

Objective: Remove the follow relation from the authenticated user to the target user and return the updated profile.

Actors: Authenticated user

Preconditions: The username path parameter is provided and an authenticated user principal is present.

Main flow:

1. The controller looks up the target user via `userRepository.findByUsername(username)` or throws `ResourceNotFoundException`.
2. The controller finds the follow relation via `userRepository.findRelation(user.getId(), target.getId())` or throws `ResourceNotFoundException`.
3. The controller removes the relation via `userRepository.removeRelation(relation)` and returns updated `ProfileData` wrapped under "profile" with HTTP 200.

Postconditions: 200 OK with a response body mapping "profile" to the updated `ProfileData`.

Possible errors: `ResourceNotFoundException`

Related endpoint:

DELETE /profiles/{username}/follow

Traceability: Controller=ProfileApi.java | Method=unfollow | Mapping=@DeleteMapping (path = "follow")

Request (Contract):

```
{
  "bodyType": null,
  "bodySchema": null,
  "parametros": {
    "path": {
      "username": "string"
    }
  }
}
```

Response (Contract):

```
{
  "responseType": "HashMap(String,ProfileData)",
  "responseSchema": null
}
```

Use case: GET /tags

Objective: Retrieve all tags used in the system.

Actors: System

Preconditions: None.

Main flow:

1. The controller calls `tagsQueryService.allTags()` and returns the result wrapped under "tags" with HTTP 200.

Postconditions: 200 OK with a response body mapping "tags" to the collection returned by `TagsQueryService`.

Possible errors: No evidence was found in the provided source code.

Related endpoint:**GET /tags**

Traceability: Controller=TagsApi.java | Method=getTags | Mapping=@GetMapping

Request (Contract):

```
{
  "bodyType": null,
  "bodySchema": null,
  "parametros": {}
}
```

Response (Contract):

```
{
  "responseType": "HashMap(String, List(String))",
  "responseSchema": null
}
```

Coverage: 17 use cases were generated for 17 endpoints (1:1).